

■ Pizzaria

Custom Pizza Ordering & Inventory Platform — Interview Ready Guide

Short & crisp answers • Project-based • System Design + Tech Deep-Dive + HR

Backend
Node + Express

Frontend
React + Vite

DB
MongoDB

Realtime
Socket.IO

Pay
Razorpay Test

Live → Frontend: pizzaria-3e3n.vercel.app | API: pizzaria-ug4h.vercel.app/api/health | GitHub: github.com/aloksingh9336/Pizzaria

How to use this guide

Read **section by section**. Each Q is what interviewers actually ask from this project. Each A is your **1–2 line** crisp answer — say it verbatim, then pause. If they probe deeper, add the *Extra line* in brackets.

Your 45-second elevator pitch (say this first)

"Pizzaria is a full-stack pizza ordering and inventory platform I built end-to-end. Users build custom pizzas through a 4-step wizard, pay via a realistic Razorpay test checkout (UPI/QR/Card/NetBanking/Wallet + demo QR), and track live delivery on a Leaflet map. Admins manage inventory with auto-decrement and low-stock email alerts via node-cron, and update order status live via Socket.IO with polling fallback. Frontend is React + Vite on Vercel, backend is Express + MongoDB Atlas, deployed serverless with SPA rewrites and monotonic order states."

Contents

1	Project Overview & Why this stack
2	Frontend (React, Vite, State, Styling)
3	Backend & API Design
4	Auth, JWT & Validation
5	Database & Data Modeling
6	Payments (Razorpay Test)
7	Realtime: Socket.IO + Polling Fallback
8	Delivery Map (Leaflet Demo)
9	Inventory & Low-Stock Cron
10	System Design & Architecture
11	Deployment (Vercel Serverless)
12	Security, Performance & Scaling
13	Challenges & Trade-offs
14	HR / Behavioral

1 — Project Overview & Why this stack

Q: What is Pizzaria in one line?

A: Full-stack pizza ordering + inventory platform: custom builder, test payments, live tracking, and admin ops — monorepo with React + Express + MongoDB.

Q: Why this stack (MERN + Socket.IO + Razorpay + Leaflet)?

A: MERN for rapid full-stack JS, Socket.IO for live order updates, Razorpay is standard in India for UPI/Cards, Leaflet+OSM for free demo maps without API keys.

Q: Why not Next.js / Firebase / Stripe?

A: Need separate admin JWT, fine-grained inventory logic, and test-mode UPI — Next.js would add SSR overhead; Stripe lacks UPI; Razorpay fits Indian context.

Q: Who are the users & roles?

A: Two roles: User (JWT user) builds & tracks pizzas; Admin (role:admin JWT, seeded only) manages inventory & orders. No public admin signup.

2 — Frontend (React, Vite, Tailwind, Router, State)

Q: Why Vite over CRA?

A: Vite = instant HMR and faster builds (~2s for 183 modules vs 30s+ CRA); simpler config for Tailwind.

Q: Why React Router + Context API (not Redux)?

A: App state is auth + cart draft (small). Context avoids Redux boilerplate; easy to upgrade to Redux Toolkit if needed.

Q: How is auth state handled?

A: AuthContext stores user + token in localStorage, axios interceptor injects Authorization header; AdminContext separate for adminToken + x-role header.

Q: How do you handle protected routes?

A: checks token/loading, redirects to /login; checks adminToken. Both in components/ProtectedRoute.jsx.

Q: How is the 4-step builder implemented?

A: Builder.jsx keeps sel {base,sauce,cheese,vegetable[]} + step index, progress bar, back/next, running price sum, out-of-stock disabled; draft saved to sessionStorage for OrderSummary.

Q: Why sessionStorage for draft?

A: Survives refresh but clears after checkout; avoids polluting DB with unpaid carts.

Q: Real images vs SVG fallback?

A: Unsplash CDN via data/images.js + ReallImage.jsx: img tag with onError fallback to SVG PizzaArt — never broken offline.

Q: Styling choice?

A: Tailwind + custom cream/red theme (CSS vars, Poppins font), card/btn utilities, keyframe animations (fadeUp, float, marquee) for premium feel.

3 — Backend & API Design

Q: Describe your REST API layout.

A: Auth: POST /api/auth/register|login, forgot/reset; Admin auth: POST /api/admin/auth/login; Menu: GET /api/pizza-options; Orders: POST /api/orders, /:id/create-payment, /:id/verify-payment, GET /my; Admin: GET|PATCH /admin/inventory, GET /admin/orders, PATCH /orders/:id/status; GET /health.

Q: Why separate user/admin auth routes & JWTs?

A: Isolation & least privilege: user token role=user, admin token role=admin; middleware authUser vs authAdmin verify role claim.

Q: Centralized error handling?

A: express-async-errors + errorHandler middleware returns {success:false, message, errors[]}; notFound for 404s; Joi validation gives uniform shape.

Q: Why express-rate-limit, helmet, cors?

A: Rate-limit auth routes (50/15min) vs brute force; helmet secures headers; cors restricts to CLIENT_URL (serverless allows *).

Q: Monorepo structure benefits?

A: Single git push deploys both; shared .env.example; clear server/src/config|models|routes|services|sockets separation.

4 — Auth, JWT & Validation

Q: JWT flow in detail?

A: bcrypt hash on save, login verifies hash, signs {id, role} with JWT_SECRET (7d). Client stores token, sends Bearer header; middleware jwt.verify. Admin login same but role=admin.

Q: Why no email verification now?

A: Removed for smoother demo — register creates isEmailVerified=true. Forgot/reset still token-based (random 32-byte hex, 1h expiry, Nodemailer console log in dev).

Q: The .local email bug you fixed?

A: Joi email() rejects non-public TLDs by default, so admin@pizzaria.local failed 400. Fixed with tlds:{allow:false} — still validates format.

Q: Joi vs Zod choice?

A: Joi is mature for Express; validates body on every POST/PATCH via validate(schema) middleware; abortEarly:false returns all errors.

Q: Password reset security?

A: Reset token stored + expiry, single-use cleared after reset, bcrypt re-hash via pre-save hook, enumeration-safe response.

5 — Database & Data Modeling

Q: Models overview?

A: User, Admin, InventoryItem (type enum, stock, threshold, lastAlertedAt), PizzaOption (category, priceModifier, linkedInventoryItemId), Order (user ref, items[], total, paymentStatus, status enum, statusHistory, razorpay ids).

Q: Why linkedInventoryItemId?

A: PizzaOption points to its InventoryItem so availability reflects live stock; fallback to name match if unlinked.

Q: How is low-stock flag computed?

A: InventoryItem.stockQuantity < lowStockThreshold → highlighted. Scanned by cron; demoed via PATCH threshold.

Q: Order statusHistory purpose?

A: Audit trail: each status change pushes {status, timestamp}; powers timeline and admin history.

Q: Indexes?

A: User/Admin email unique; PizzaOption unique on (category,name); InventoryItem unique name — prevents duplicates via upsert seed.

6 — Payments (Razorpay Test) & Realistic Checkout

Q: How does test payment work without real keys?

A: RAZORPAY MOCK=true skips Razorpay Orders API: create mock razorpayOrderId order_mock_, frontend simulates collectors.

Q: What does the realistic checkout add?

A: RazorpayCheckout.jsx mimics real Razorpay: left amount summary, right tabs UPI / QR / Card / NetBanking / Wallet, validation, processing spinner + progress bar, success bounce check.

Q: UPI & QR demo details?

A: UPI ID regex + one-tap GPay/PhonePe/Paytm tiles; QR encodes upi://pay?pa=pizzaria@razorpay&am=... via api.qrserver.com, 5:00 countdown, Simulate Success/Failure.

Q: Real Razorpay path?

A: If mock=false + keys set + window.Razorpay present, opens real Razorpay Checkout with handler verifying signature via HMAC SHA256 vs RAZORPAY_KEY_SECRET.

Q: How is payment verified & stock updated?

A: POST /:id/verify-payment checks signature (or simulateSuccess flag), sets paid, calls decrementForOrder, emits order:created + order:status.

7 — Realtime: Socket.IO + Polling Fallback

Q: Why Socket.IO, not pure WebSockets?

A: Rooms (user:<id>, admin), auto-reconnect, fallback transport, easy emit to specific user.

Q: Join flow?

A: Client emits join {token|adminToken}, server jwt.verify, joins room; on payment/status update server emits order:status to user: and admin.

Q: Why polling fallback?

A: Vercel serverless kills long websockets. Dashboard polls every 8s, Tracking every 6s — live updates work even without socket.

Q: Admin status update live?

A: PATCH /admin/orders/:id/status updates status + statusHistory, emits to user: & admin — user's Dashboard card re-renders instantly.

8 — Delivery Map (Demo Leaflet)

Q: Why Leaflet + OSM vs Google Maps?

A: Free, no API key, works on Vercel, demo-only; Google needs billing and key management.

Q: How is the rider animated?

A: Bezier curve STORE (28.6139,77.2090) → CUSTOMER, 40 points, sliced by progress t (0→1). When Sent to Delivery, interval animates 0→1 over 90s, updates ETA & polyline.

Q: What's demo vs real?

A: All fake: static Delhi coords, random partner (pravatar), tel: link, simulate next status button. No real GPS.

Q: Why not persist movement?

A: Serverless can't hold state; map is client-side simulation driven by order.status.

9 — Inventory & Low-Stock Cron

Q: Auto-decrement logic?

A: decrementForOrder counts names from items (base,sauce,cheese,vegs) case-insensitive, finds InventoryItem by name regex, decrements by 1 (min 0).

Q: Manual update?

A: PATCH /admin/inventory/:id {stockQuantity, lowStockThreshold} — inline table edit, emits stock:updated to admin room.

Q: Low-stock email flow?

A: node-cron per STOCK_CRON_SCHEDULE, finds stock<threshold, skips if lastAlertedAt within COOLDOWN, sends via Nodemailer, sets lastAlertedAt, emits stock:low.

Q: Why console-stream in dev?

A: SMTP_HOST empty → streamTransport logs to console so demo works without Ethereal/Gmail; links + tokens printed + returned in dev responses.

Q: Vercel limitation?

A: Cron doesn't persist serverless; use npm run trigger:stock-check locally or move backend to Render for real cron.

10 — System Design & Architecture

Q: Draw the architecture in 30 seconds.

A: Vercel Frontend (Vite SPA, React Router, Rewrites) → Axios → Vercel Serverless API (api/index.js, Express) → Mongoose → MongoDB Atlas. Local dev variant adds http+Socket.IO rooms & node-cron.

Q: Why serverless for API?

A: Free Vercel deploy, auto-scale, but REST only — websockets fall back to polling. For full realtime move to Render/EC2.

Q: Data flow for an order?

A: Builder→sessionStorage→POST /orders (pending)→create-payment→RazorpayCheckout→verify-payment→decrement→socket emit→Tracking+Map & Admin refresh.

Q: 5 bases & 5 sauces requirement?

A: Seed enforces 5 bases (Thin, Thick, Cheese Burst, Whole Wheat, Gluten-Free) & 5 sauces (Tomato Basil, BBQ, Pesto, Alfredo, Peri Peri) per checklist.

11 — Deployment (Vercel)

Q: Two projects from one repo?

A: Project A: Root server, vercel.json rewrites → /api. Project B: Root client, Framework Vite, Output dist, vercel.json rewrites → index.html for SPA.

Q: Env vars needed?

A: Backend: MONGO_URI, JWT_SECRET, CLIENT_URL (frontend URL), RAZORPAY MOCK, SMTP_*. Frontend: VITE_API_URL (=backend/api), VITE_SOCKET_URL (=backend).

Q: SPA 404 on /admin/login you fixed?

A: Direct URL hit returned 404_NOT_FOUND; fixed with client/vercel.json rewrites to index.html so React Router handles it.

Q: Backend fsPath build error you fixed?

A: Deprecated builds/routes in vercel.json caused Cannot read fsPath; replaced with rewrites.

Q: DNS loopback bug?

A: Node saw DNS 127.0.0.1 (VPN) → ECONNREFUSED for Atlas SRV; fixed in config/db.js by falling back to 8.8.8.8/1.1.1.1.

12 — Security, Performance & Scaling

Q: Security checklist?

A: bcrypt hashing, JWT 7d, helmet, cors whitelist, rate-limit auth, Joi stripUnknown, no admin public register, .env gitignored, .env.example committed.

Q: Performance wins?

A: Vite build ~2s/183 modules, lazy images, polling backoff, small Context vs Redux, Tailwind purge, leaflet CSS 45KB.

Q: How to scale?

A: Horizontally: stateless API + Atlas + Vercel edge; add Redis for rate-limit & socket adapter, queue for emails, paginate /admin/orders, index Order.user+status.

Q: What would you add next?

A: Redis cache for pizza-options, order pagination, image CDN + compression, Sentry logging, e2e tests with Playwright, role-based rate limits.

13 — Challenges & Trade-offs

Q: Toughest bug?

A: Atlas DNS SRV ECONNREFUSED due to loopback-only resolver; diagnosed via nslookup vs Node dns, fixed with public DNS fallback.

Q: Trade-off: real images vs bundle size?

A: Chose Unsplash CDN + lazy + SVG fallback over bundling 18 images — saves bundle, needs internet.

Q: Trade-off: serverless vs always-on Node?

A: Vercel is free & fast but no persistent websockets/cron; accepted polling fallback, noted Render alternative for full realtime.

14 — HR / Behavioral (say STAR: Situation-Task-Action-Result)

Q: Tell me about this project.

A: Built a production-grade full-stack pizza platform from scratch to last spec — owned planning, schema, APIs, UI, theme, payments, map, deploy. Shipped live on Vercel with CI via GitHub.

Q: A time you debugged under pressure?

A: Admin login 400 Validation failed on deploy: traced to Joi TLD check blocking .local domain, patched and redeployed in minutes.

Q: How do you handle feedback/changes?

A: Used staged todos, built feature-by-feature per checklist, synced row/column polish and theme iterations via small commits; kept README as living doc.

Q: Why should we hire you?

A: I ship end-to-end, handle ambiguous specs, debug infra (DNS, Vercel rewrites) and product polish (animations, real images) equally; clean commits and docs make handover easy.

Q: Where do you see gaps?

A: Want deeper testing (Jest/Playwright), observability, and payment webhook reconciliation — next steps on my roadmap.

Pro tips for your interview: 1) Start with the 45-sec pitch. 2) For any “how” question, name the file:line (e.g. *RazorpayCheckout.jsx*, *services/inventoryService.js*). 3) Always mention the fallback: “Socket.IO with polling fallback on Vercel” — shows you know limits. 4) End each answer with a live demo link.

Generated for you — Pizzaria • 2026-09-12 • Keep this to 1 print; speak crisp, not long.